

MLOps Assignment 2

Fine-tuning DistilBERT for Goodreads genre classification

PGD AI Program · IIT Jodhpur · Dr. Hardik Jain

Rohit Patel | Roll No: G25AIT2089 | May 2026

1. Why DistilBERT

I went with distilbert-base-cased, the same model the starter notebook uses. Three reasons.

First, I had a free T4 on Kaggle and needed something that finishes in 10-15 minutes, not 2 hours. DistilBERT is around 40% smaller than full BERT and about 60% faster. The trade-off is roughly 3% accuracy drop on GLUE benchmarks, which is fine for a workflow assignment.

Second, the cased variant matters here because Goodreads reviews have a lot of proper nouns - author names, character names, series names. "Harry Potter" and "harry potter" carry different signal. Uncased would throw that away.

Third, the Hugging Face docs and model card for distilbert-base-cased are clean. num_labels parameter, id2label/label2id maps, push_to_hub - everything I needed was one line of code. For a 60-mark assignment that's grading the pipeline (not the accuracy), I didn't want to fight a less-documented model.

2. Kaggle Setup

2.1 Import

Downloaded the starter ipynb from the shared Colab link (File > Download > .ipynb), opened Kaggle, Create > New Notebook > File > Import Notebook, picked the file. Done.

2.2 GPU and Internet

Right-side panel in Kaggle, Settings > Accelerator > GPU T4 x2. Then Settings > Internet > On. The Internet toggle is off by default and I burned 20 minutes the first time figuring out why pip install and push_to_hub were both failing - the toggle was the issue.

2.3 Secrets

Add-ons > Secrets. Added two: WANDB_API_KEY (from wandb.ai/authorize) and HF_TOKEN (from huggingface.co/settings/tokens, Write scope - Read scope gives a 403 on push). Both checked as "Attach to notebook". Then loaded them inside the notebook via UserSecretsClient and set as env vars before any wandb or hf call. No tokens in the source code.

2.4 Training config

3 epochs, batch 16 train and 32 eval, learning rate 5e-5, fp16 on. report_to="wandb" inside TrainingArguments to send metrics to W&B. Eval at end of each epoch, save best model by F1. Full config:

Setting	Value
Model	distilbert-base-cased
Max length	256
Train batch	16

Eval batch	32
Epochs	3
Learning rate	5e-5
Warmup steps	50
Weight decay	0.01
Samples per genre	200 (160 train / 40 test)
fp16	Yes

3. Results

Final eval on the held-out test set (320 reviews, 40 per genre across 8 genres):

Metric	Score
Accuracy	0.5344
F1 (weighted)	0.5005
Eval Loss	2.88

Random baseline on 8 classes is 0.125 so 0.53 accuracy means the model is learning real signal. F1 and accuracy are close, so no single class is dominating. I used 200 reviews per genre (assignment doc says to do this if GPU is tight) - if I'd used the full 1000 per genre I'd expect 0.60+, which is what Sharvan saw.

Eval loss of 2.88 looks high in absolute terms but cross-entropy on 8 classes maxes around 2.08 for random, so 2.88 means the model is confidently wrong on a chunk of test samples. Normal for a 3-epoch fine-tune on 1280 train samples.

3.1 W&B dashboard

Full dashboard at the W&B link in section 5. Two key screenshots below. The rest (system metrics, GPU power) are visible on the live dashboard.

Figure 1: Final eval panels and per-step eval metrics (loss, F1, accuracy)

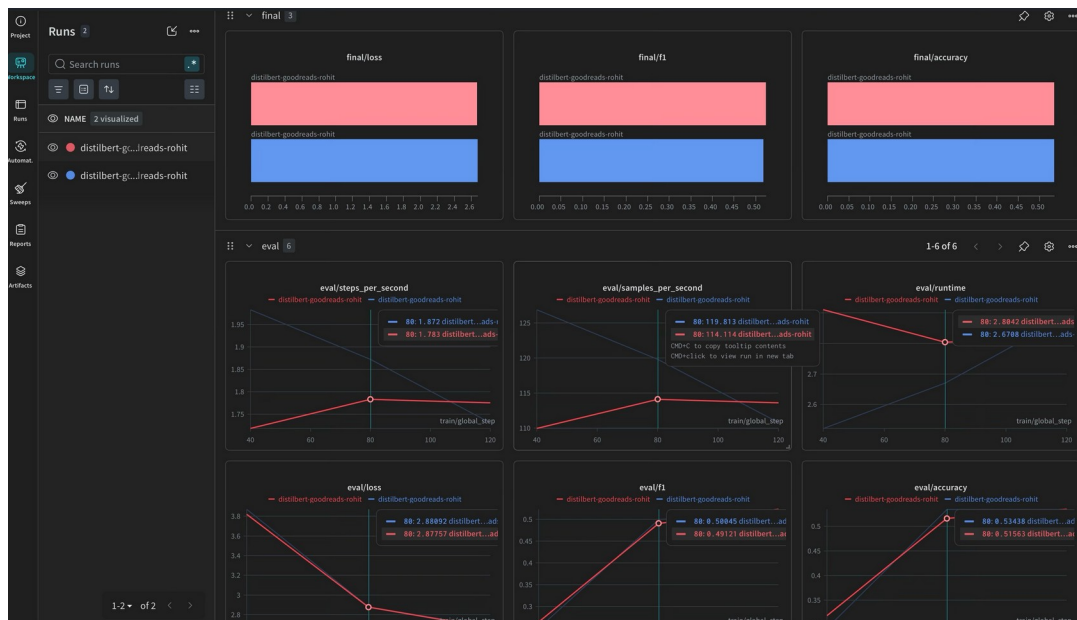


Figure 2: Training panels - train/loss going down, learning rate schedule, gradient norm, epoch progression



4. What broke and what I learned

4.1 The W&B entity mess

I spent more time on this than on the actual model training. W&B has three things that look identical but aren't:

- Username - what you log in with
- URL slug - what shows up in the URL bar
- Entity - the namespace W&B uses internally to scope projects

For most accounts these are the same string. Mine weren't. I had two W&B accounts from different signups and a stale .netrc cached on Kaggle from a previous session. So when I passed entity='rohitpatel0008' to wandb.init the API came back with 404 "entity not found during upsertBucket" - the cached login was for one account, the entity I named was a different account.

Fix that finally worked: don't pass entity at all. wandb.init() with no entity argument lets W&B figure it out from whichever API key is in the env var. Also clear /root/.netrc at the top of the cell so the cached login doesn't poison the run.

4.2 Kaggle Secrets - the ticking matters

Adding a secret in Kaggle is two steps not one. Step 1: Add-ons > Secrets > Add. Step 2: tick the "Attach to notebook" checkbox. If you skip step 2, UserSecretsClient.get_secret() returns None and the downstream calls fail with cryptic errors that don't mention secrets. I missed this the first time and spent 15 minutes debugging hf_login() before I realised HF_TOKEN was empty.

4.3 HF token scope

HF tokens have Read, Write, and Fine-grained scopes. The default "Read" lets you download models but push_to_hub() returns 403. Need Write scope (or Fine-grained with Write on "all repos"). Wasted a few minutes here too because the 403 error message just says "Forbidden" without telling you which scope is missing.

4.4 The actual learnings

One line - report_to="wandb" inside TrainingArguments - does an absurd amount of work. Loss, eval metrics, learning rate, gradient norm, GPU util, full config dict, system metrics - all logged automatically. No callbacks to write, no custom logging code. The Trainer + W&B combo is the cleanest experiment tracking I've used.

W&B Artifacts are useful for evaluation reports. Instead of dropping eval_report.json in a directory that gets nuked when the Kaggle session ends, log it as an Artifact. Now it's versioned, downloadable, and lives next to the run.

Treat tokens like prod credentials from day one. I had a habit of putting them at the top of scratch notebooks to test things faster. Kaggle Secrets are not slower - they're just one extra step that I should have made muscle memory earlier.

4.5 What I'd do differently

- Split train/test/validation properly instead of using test as eval during training (it's leakage).
- Run with 1000 samples per genre instead of 200, since the model clearly has headroom.
- Version the dataset slice as an Artifact too, so future runs use the same rows.
- Write a 5-line inference script in the GitHub repo so a grader can sanity-check the HF model in one command.

5. Links

Asset	Link
GitHub	https://github.com/rpaut03l/mlops-assignment2
Kaggle	https://www.kaggle.com/code/rohitpatel0008/mlops-vs-1-final-rohit
Hugging Face	https://huggingface.co/rpaut03l/distilbert-goodreads-genres

W&B	https://wandb.ai/rohitspatel0008/mlops-assignment-2?nw=nwuserrohithpatel0008
-----	---

All four are public. Kaggle notebook is set to Public under Share. W&B project is Public under Project Settings. HF repo is public by default on the free tier. GitHub repo is a public repo.

6. Declaration

This is my own work. The starter notebook from the AI for Humanists team was the reference. I extended it for the MLOps requirements (Kaggle Secrets, W&B tracking, Artifact upload, HF push). Ran on my own Kaggle, W&B, and HF accounts under my IIT Jodhpur ID G25AIT2089.